

Software Architecture and Design Specification

Project: Snake Game

Version: 1.0

Authors: Team 15

Date: 14-09-2026

Status: Draft

Revision History

Version | Date | Author | Change Summary

1.0 | 14-09-2026 | Team 15 | Initial architecture and design derived from the Snake Game SRS.

Approvals

Role | Name | Signature/Date

Course Coordinator | |

1. Introduction

1.1 Purpose

This document specifies the architecture and design of the Snake Game, a single-player game where the player steers a growing snake around a fixed grid, eats food to score, and loses on hitting a wall or itself.

1.2 Scope

Covers single-player gameplay: main menu, game loop (movement, food, collision, scoring), game-over flow, and a locally stored high score. Excludes multiplayer, online leaderboards, and in-app purchases.

1.3 Audience

Developers building the game, testers checking it against the SRS, and reviewers or instructors assessing the project.

1.4 Definitions

FPS, Tick, Tick rate, FR, NFR, ADR, UI.

2. Document Overview

2.1 How to use this document

Provides architectural deliverables including UML diagrams, module descriptions, threat model, and internal API design.

2.2 Related Documents

SRS (Snake Game v1.0), STP, RTM.

3. Architecture

3.1 Goals & Constraints

Goals: 60 FPS on typical hardware, key press reflected within one tick, no crashes in a normal session, input/logic/rendering in separate modules. Constraints: fixed grid size and starting speed for v1.0, no network required, game reads/writes only its own save file.

3.2 Stakeholders & Concerns

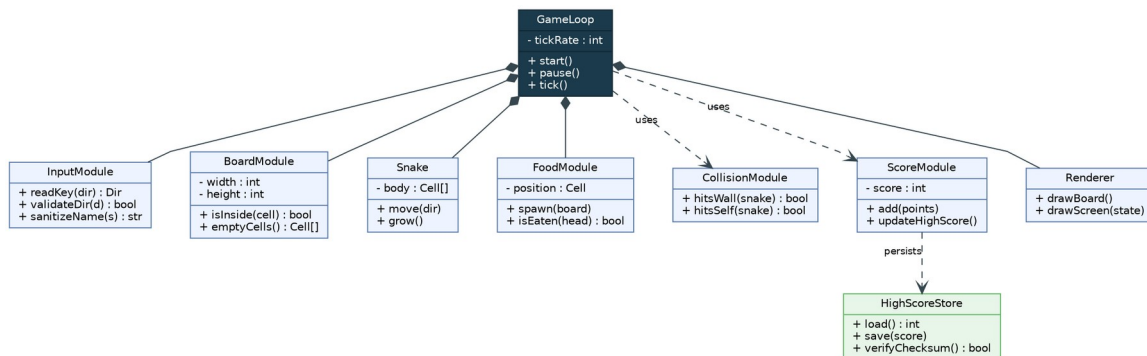
Player: responsiveness, steady frame rate, no crashes, persistent high score

Developer: clear separation of input, logic, rendering

Tester: every requirement traceable to a module and test case

Reviewer: design that follows from the SRS.

3.3 Component (UML) Diagram



3.4 Component Descriptions

- **GameLoop**: runs each tick (read input, move snake, check collisions, update score, redraw); speeds up as score crosses thresholds.
- **InputModule**: turns key presses into a direction, rejects reversal into self, sanitizes player-name input.
- **BoardModule**: holds the fixed grid, checks whether a cell is inside the board, lists empty cells for food.
- **Snake**: body as a list of cells; moves one cell per tick and grows on eating.
- **FoodModule**: places food on a random empty cell and reports when the head reaches it.
- **CollisionModule**: detects wall and self collisions and signals game-over.
- **ScoreModule**: adds points per food and decides when the high score updates.
- **HighScoreStore**: reads/writes the high score to a local file with a checksum to detect

tampering.

- Renderer: draws board, snake, food, score, and the menu and game-over screens.

3.5 Chosen Architecture Pattern and Rationale

Layered architecture with a central game-loop controller: input, logic/domain, and presentation layers coordinated each tick. Satisfies the requirement to keep input, logic, and rendering in separate modules. Event-driven/ECS and microservice designs rejected as heavier than a single-player offline game needs.

3.6 Technology Stack & Data Stores

Desktop: a 2D framework such as Python/Pygame or C++/SDL. Web: HTML5 Canvas with JavaScript. Data store: a single local save file (JSON/binary on desktop, localStorage in browser) holding only the high score and checksum. No database, no network for core gameplay.

3.7 Risks & Mitigations

Risk: game speed varies with hardware -> Mitigation: drive movement off a fixed tick interval, not the frame rate.

Risk: rapid key presses cause a crash or illegal reversal -> Mitigation: one validated direction change per tick.

Risk: save file hand-edited or corrupted -> Mitigation: checksum with fallback to a default value.

Risk: malicious text in the name field -> Mitigation: strip/reject HTML and script content, cap length.

3.8 Traceability to Requirements

SNK-F-002 (Create board) -> BoardModule

SNK-F-005 (Change direction) -> InputModule

SNK-F-007 (Spawn food) -> FoodModule

SNK-F-010 (Self-collision) -> CollisionModule

SNK-F-014 (Save high score) -> ScoreModule + HighScoreStore

SNK-NF-001 (60 FPS loop) -> GameLoop

SNK-SR-001 (Sanitize name input) -> InputModule

3.9 Security Architecture

Threat Modeling (STRIDE): Spoofing -> not applicable (no accounts in v1.0); Tampering -> checksum on save file; Info disclosure -> only a high score stored, TLS if a future leaderboard is added; DoS -> one validated direction change per tick; Elevation of Privilege -> file access restricted to the game's own save path; Injection -> strip/reject markup in the name field.

4. Design

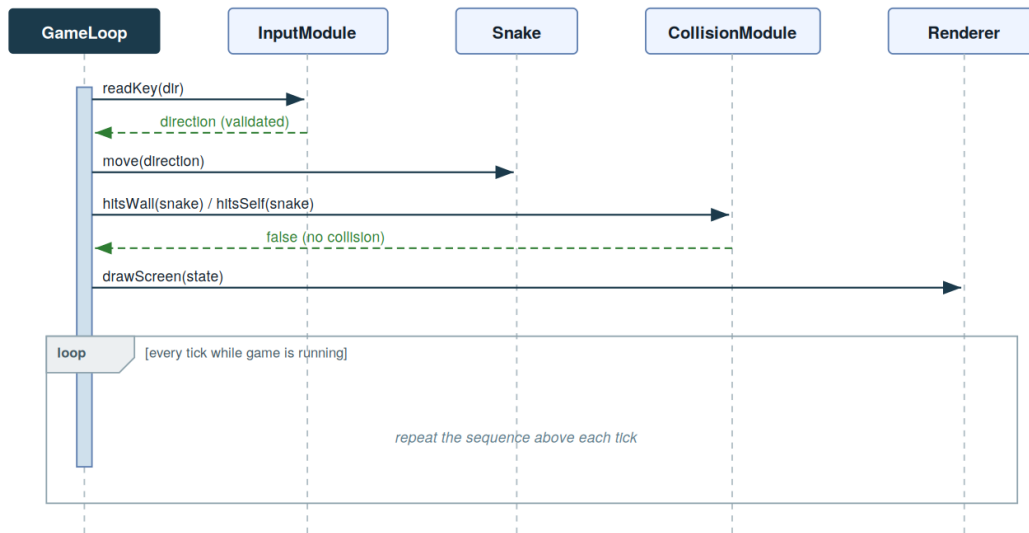
4.1 Design Overview

Layered modules coordinated by GameLoop so input, logic, and rendering are separate. Each tick runs a fixed order: read input, move snake, resolve food, check collisions, update score, render.

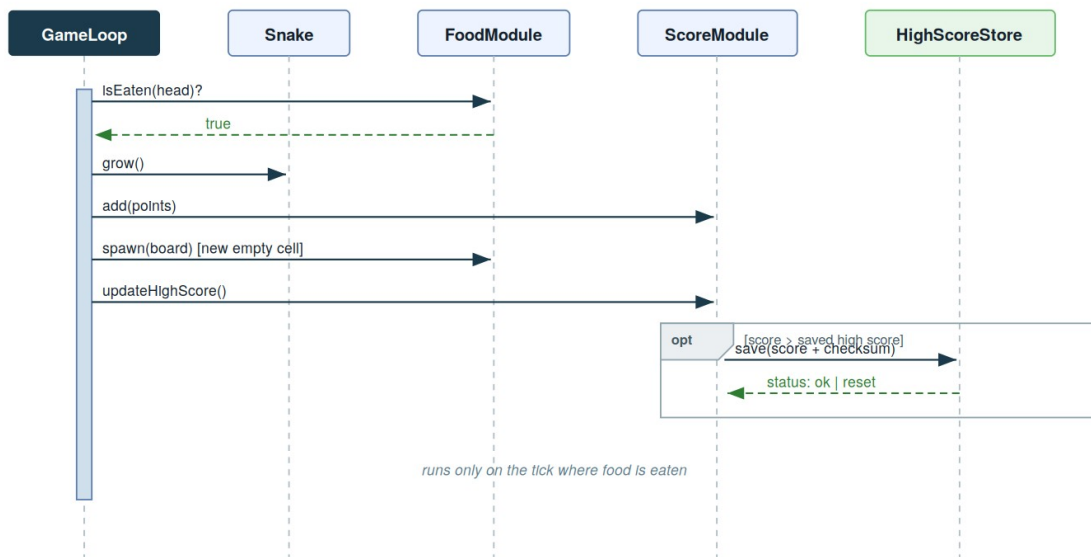
4.2 UML Sequence Diagrams

Two flows specific to this project: (1) one game tick (movement and control), and (2) eating food and persisting the high score.

Flow 1: One game tick (movement and control).



Flow 2: Eating food and persisting the high score.



4.3 API Design

Interface definition for at least two modules (the game has no network API, so these are internal module interfaces).

Call: `InputModule.readKey(currentDir)`

Request: `currentDir`

Returns: `{direction}` (validated new heading)

Rejects: a reversal directly into the snake (returns `currentDir` unchanged)

Call: `HighScoreStore.save(score)`

Request: `{score}`

Response: `{status: ok | reset}`

Errors: checksum mismatch on existing file -> status reset, value falls back to default (0)

4.4 Error Handling, Logging & Monitoring

Game degrades rather than crashes: an unreadable or checksum-failed save resets the high score; an invalid direction is ignored. No player-entered text is logged verbatim. During development, track average frame time against the ~16.7 ms budget and input-to-movement latency within one tick.

4.5 UX Design

Three screens: main menu (Start, High Score, Exit), gameplay (grid, snake, food, live score), and game-over (final score with restart). Controls: arrow keys or WASD to steer, one key to pause/resume, one to confirm/restart. Large, legible text.

4.6 Open Issues & Next Steps

Future enhancements: configurable board size and speed, online leaderboard (over HTTPS/TLS), mobile build with touch controls, and finalising the speed-up score thresholds.

5. Appendices

5.1 Glossary: FPS, Tick, Tick rate, FR, NFR, ADR.

5.2 References: SRS Snake Game v1.0, IEEE 42010, OWASP.

5.3 Tools: PlantUML, draw.io, and the chosen 2D framework tooling (Pygame/SDL or HTML5 Canvas).